

Deep Learning for Solving Dynamic Models

Session 5 — Scaling up: the international real business cycle model

Simon Scheidegger

HEC, University of Lausanne

University of Padova

September 23, 2026

Based on: Azinovic, Gaegauf & Scheidegger (2022), *International Economic Review* 63(4), 1471–1525.
Brumm, Krause, Schaab & Scheidegger (2022), “Sparse Grids for Dynamic Economic Models,” *Oxford Research Encyclopedia of Economics and Finance*, Oxford University Press.

Roadmap of This Lecture

Part A

- ▶ **Part I.** The IRBC model: preferences, production, TFP, adjustment costs, irreversibility, Fischer–Burmeister
- ▶ Equilibrium system + finger exercise

Part B

- ▶ **Part II.** DEQN formulation: states, architecture, loss, Gauss–Hermite, training algorithm
- ▶ **Part III.** Implementation: Keras code, convergence, policy functions, validation
- ▶ Bridge to the production-code DEQN library

Learning objectives

By the end of this lecture you should be able to:

- ▶ Set up the International Real Business Cycle model with multi-country state space, capital adjustment costs, and irreversibility constraints
- ▶ Derive first-order conditions (Euler equations, consumption sharing) and convert KKT conditions into smooth complementarity functions (Fischer–Burmeister)
- ▶ Formulate the DEQN loss function for multi-country DSGE models and train the network on simulated paths
- ▶ Implement policy functions in Keras and validate numerical solutions against theoretical steady states
- ▶ Extend the framework to 4–100+ dimensional problems and perform sensitivity analysis on model parameters

Part I

The IRBC Model

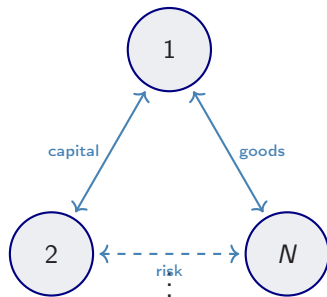
Why International Models?

Key questions in international macro:

- ▶ How do **capital flows** respond to productivity shocks?
- ▶ What governs **international risk sharing**?
- ▶ How do **trade imbalances** emerge?

⇒ Need models with N heterogeneous countries.

⇒ State space: $2N$ dimensions (capital + TFP per country).



Complete markets

Why IRBC for Macro-Finance Research?

Workhorse for **open-economy asset pricing** and **international risk sharing**:

- ▶ **International risk sharing.** Complete markets imply perfectly correlated consumption; data do not (Backus, Kehoe & Kydland 1992).
- ▶ **Market structure & welfare.** Complete vs. incomplete markets; welfare cost of missing insurance (Heathcote & Perri 2002).
- ▶ **Capital flows & current account** from intertemporal savings + heterogeneous productivity.
- ▶ **Home bias:** frictionless benchmark for observed portfolio holdings.
- ▶ **Macro-financial transmission** via adjustment costs, constraints, Pareto weights.

IRBC: Puzzles and Computational Testbed

Macro-finance puzzles:

- ▶ **Consumption-correlation puzzle**
(Backus–Kehoe–Kydlund 1992): complete markets imply $\rho(c^i, c^j) \approx 1$, but in the data $\rho(c^i, c^j) < \rho(y^i, y^j)$.
- ▶ **Portfolio home-bias puzzle**: optimal diversification predicts large foreign-equity holdings; observed portfolios are strongly domestic.
- ▶ **Real-exchange-rate (RER) volatility**
(Backus–Smith): the ratio of price levels P^j/P^i is far more volatile than the consumption ratio c^j/c^i ; IRBC with nontraded goods (Stockman & Tesar, 1995) reproduces this.

Computational testbed:

- ▶ State dim. **linear** in N : $2N$.
- ▶ Smooth but **highly nonlinear**.
- ▶ **Planner** gives a clean target.
- ▶ Benchmark for projection methods (Krueger & Kubler 2004; Brumm & Scheidegger 2017).

Model Overview

International Real Business Cycle (IRBC) model — Backus, Kehoe & Kydland (1992); here with the DEQN-style extensions of Azinovic, Gaegauf & Scheidegger (2022):

- ▶ N countries, each with **country-specific capital** k^j and **TFP** z^j
- ▶ **Complete markets**: a social planner allocates resources with Pareto weights τ^j
- ▶ **Irreversible investment**: $I^j \geq 0$ (cannot disinvest)
- ▶ **Capital adjustment costs**: $\Gamma^j = \frac{\kappa}{2} k^j \left(\frac{k^{j'}}{k^j} - 1 \right)^2$

Dimensionality

State: $\mathbf{s} = (k^1, \dots, k^N, z^1, \dots, z^N) \in \mathbb{R}^{2N}$

Policy: $(k^{1'}, \dots, k^{N'}, \lambda, \mu^1, \dots, \mu^N) \in \mathbb{R}^{2N+1}$

Preferences

Each country j has **CRRA utility** (with the standard log limit at $\gamma_j = 1$):

$$u^j(c) = \begin{cases} \frac{c^{1-1/\gamma_j} - 1}{1 - 1/\gamma_j}, & \gamma_j \neq 1, \\ \ln c, & \gamma_j = 1. \end{cases}$$

- ▶ γ_j : intertemporal elasticity of substitution for country j
- ▶ **Heterogeneous** across countries: γ_j linearly spaced in $[0.25, 1.0]$ (so risk aversion $1/\gamma_j \in [1, 4]$; the $\ln c$ branch is used at $\gamma_j = 1$)

Notation: in this chapter γ is the **IES**; later chapters on continuous-time HA and climate re-purpose γ for CRRA risk aversion (with ψ then the IES). Restated at the top of each chapter.

Social planner maximizes with Pareto weights τ^j :

$$\max \mathbb{E}_0 \left[\sum_{t=0}^{\infty} \beta^t \sum_{j=1}^N \tau^j u^j(c_t^j) \right]$$

Production

Each country j produces a single good:

$$Y^j = A_{\text{tfp}} \cdot \exp(z^j) \cdot (k^j)^\zeta$$

Calibration of A_{tfp} : chosen so that the **steady state** is consistent with $k_{ss} = 1$:

$$A_{\text{tfp}} = \frac{1 - \beta(1 - \delta)}{\zeta \cdot \beta}$$

Parameter	Value
ζ (capital share)	0.36
β (discount factor)	0.99
δ (depreciation)	0.01

TFP Process

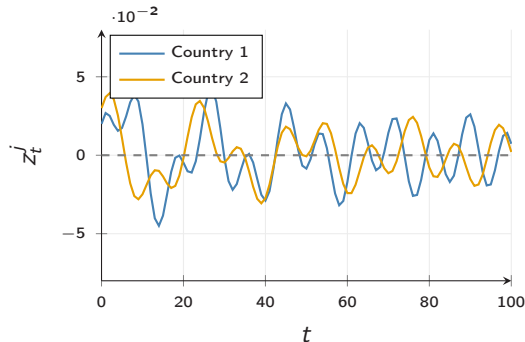
Log TFP follows an **AR(1)** with common and idiosyncratic shocks:

$$z^{j'} = \rho_z z^j + \sigma_e (\varepsilon^j + \varepsilon^{\text{agg}})$$

- ▶ $\varepsilon^j \sim \mathcal{N}(0, 1)$: country-specific
- ▶ $\varepsilon^{\text{agg}} \sim \mathcal{N}(0, 1)$: aggregate shock
- ▶ Total: $N + 1$ independent shocks

$$\rho_z = 0.95, \sigma_e = 0.01$$

σ_e is the *per-component* std., so the per-country innovation std. is $\sigma_e \sqrt{2}$ and the cross-country innovation correlation is $1/2$.



Stylized TFP sample paths

Capital Adjustment Costs

Changing the capital stock is **costly**
($\kappa = 0.5$):

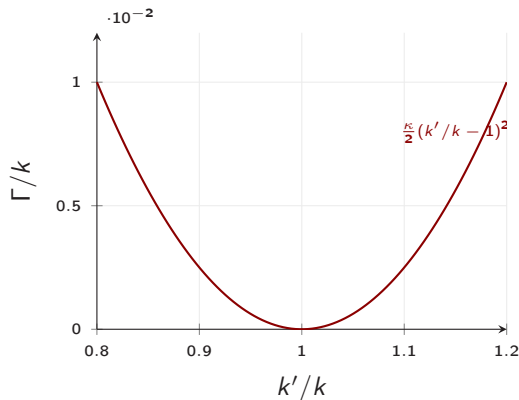
$$\Gamma^j = \frac{\kappa}{2} k^j \left(\frac{k^{j'}}{k^j} - 1 \right)^2$$

Key derivatives:

$$\frac{\partial \Gamma}{\partial k'} = \kappa \left(\frac{k'}{k} - 1 \right)$$

$$\frac{\partial \Gamma}{\partial k} = -\frac{\kappa}{2} \left(\frac{k'}{k} - 1 \right) \left(\frac{k'}{k} + 1 \right)$$

Role: smooth investment, break Tobin's $q=1$,
and dampen cross-country **capital flows**.



Adjustment cost per unit of capital

Putting It Together: The Complete Model

Primitives (from the previous slides):

Preferences: $U^j(c) = \frac{c^{1-1/\gamma_j}-1}{1-1/\gamma_j}$

Production: $Y_t^j = A_{\text{tfp}} \exp(z_t^j) (k_t^j)^\zeta$

TFP process: $z_{t+1}^j = \rho_z z_t^j + \sigma_e (\varepsilon_{t+1}^j + \varepsilon_{t+1}^{\text{agg}})$

Adjustment cost: $\Gamma_t^j = \frac{\kappa}{2} k_t^j (k_{t+1}^j/k_t^j - 1)^2$

Constraints:

Aggregate resource (ARC):
$$\sum_{j=1}^N \left[Y_t^j + (1 - \delta) k_t^j - k_{t+1}^j - \Gamma_t^j - c_t^j \right] = 0$$

Irreversibility:
$$l_t^j \equiv k_{t+1}^j - (1 - \delta) k_t^j \geq 0 \quad \forall j = 1, \dots, N$$

The planner chooses $\{c_t^j, k_{t+1}^j\}_{j,t}$ subject to these constraints.

Planner's Problem (Dynamic Optimization)

$$\max_{\{c_t^j, k_{t+1}^j\}} \mathbb{E}_0 \left[\sum_{t=0}^{\infty} \beta^t \sum_{j=1}^N \tau^j \frac{(c_t^j)^{1-1/\gamma_j} - 1}{1 - 1/\gamma_j} \right]$$

subject to:

$$\text{ARC: } \sum_{j=1}^N \left[Y_t^j + (1 - \delta) k_t^j - k_{t+1}^j - \Gamma_t^j - c_t^j \right] = 0$$

$$\text{Irreversibility: } k_{t+1}^j - (1 - \delta) k_t^j \geq 0 \quad \forall j$$

$$\text{Production: } Y_t^j = A_{\text{tfp}} \exp(z_t^j) (k_t^j)^\zeta$$

$$\text{TFP: } z_{t+1}^j = \rho_z z_t^j + \sigma_e (\varepsilon_{t+1}^j + \varepsilon_{t+1}^{\text{agg}})$$

Pareto weights τ^j allocate welfare across countries. We now *attach multipliers* to the constraints to obtain the first-order conditions.

The Lagrangian: Attaching Multipliers

Attach one multiplier per active constraint of the planner's problem:

- ▶ λ_t on the aggregate resource constraint — the **shadow price of world output** at date t .
- ▶ $\mu_t^j \geq 0$ on country j 's irreversibility — the **shadow price of being unable to disinvest**.

$$\mathcal{L} = \mathbb{E}_0 \left[\sum_{t=0}^{\infty} \beta^t \left(\sum_j \tau^j \frac{(c_t^j)^{1-1/\gamma_j}-1}{1-1/\gamma_j} + \lambda_t \sum_j (Y_t^j + (1-\delta)k_t^j - k_{t+1}^j - \Gamma_t^j - c_t^j) + \sum_j \mu_t^j (k_{t+1}^j - (1-\delta)k_t^j) \right) \right]$$

Economic reading:

- ▶ λ_t high $\Rightarrow$ world resources scarce $\Rightarrow$ every country cuts consumption.
- ▶ $\mu_t^j > 0$ only when country j 's irreversibility constraint *binds*.

KKT Conditions for Irreversibility

With the multipliers μ_t^j now defined, optimality of the planner's choice of k_{t+1}^j together with $\mu_t^j \geq 0$ yields the **complementary slackness** structure:

$$\mu^j \geq 0, \quad I^j \geq 0, \quad \mu^j \cdot I^j = 0.$$

Economic reading:

- ▶ $I^j > 0$ (agent invests): constraint slack $\Rightarrow \mu^j = 0$.
- ▶ $I^j = 0$ (constraint binds): $\mu^j \geq 0$ records the shadow value of enforcing $I^j \geq 0$.

The non-negativity $\mu^j \geq 0$ is the *standard KKT sign*: the multiplier on a $\geq$ -constraint is non-negative at the optimum. The smoothed Fischer–Burmeister residual on the next slide encodes this sign restriction *and* the slackness condition $\mu^j I^j = 0$ in one differentiable expression.

Numerical problem: KKT involves min / max — **non-differentiable**, incompatible with SGD. Next slide: smooth it via the Fischer–Burmeister function.

Fischer–Burmeister Complementarity Function

Problem: KKT conditions involve **non-smooth** min/max operators.

Solution: use a smoothed **Fischer–Burmeister** (FB) residual:

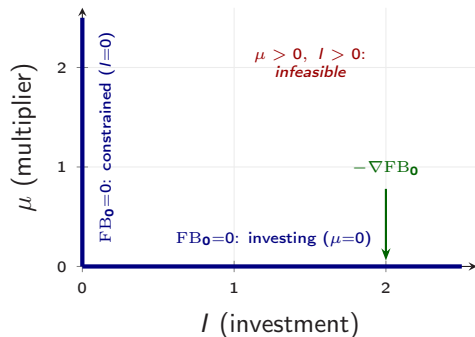
$$\text{FB}_\varepsilon(\mu, I) = \mu + I - \sqrt{\mu^2 + I^2 + \varepsilon^2}$$

$\varepsilon > 0$ small for numerical stability.

$$\text{FB}_0(\mu, I) = 0 \Leftrightarrow \mu \geq 0, I \geq 0, \mu \cdot I = 0.$$

For $\varepsilon > 0$, this exact zero set is slightly smoothed near the origin.

$\Rightarrow$ For $\varepsilon > 0$, **differentiable** everywhere—compatible with SGD.



$\text{FB}_0=0$ exactly on the two blue half-axes (one of μ, I is zero, the other ≥ 0). The open quadrant is infeasible; $-\nabla \text{FB}_0$ (green) pushes a prediction there back to the nearest axis.

FOC: Consumption Sharing

First-order condition w.r.t. c^j :

$$\tau^j \cdot (c^j)^{-1/\gamma_j} = \lambda$$

where λ is the **shadow price** of the aggregate resource constraint.

Solving for consumption:

$$c^j = \left(\frac{\lambda}{\tau^j} \right)^{-\gamma_j}$$

- ▶ λ is the **single** policy variable governing consumption allocation
- ▶ Higher $\lambda \Rightarrow$ resources are scarcer $\Rightarrow$ lower consumption
- ▶ Countries with higher γ_j (lower risk aversion) respond more to λ

FOC: Euler Equation

First-order condition w.r.t. $k^{j'}$:

$$\lambda (1 + \Gamma_{k'}) = \beta \mathbb{E}[\lambda' \cdot \text{MPK}^{j'} - (1 - \delta) \mu^{j'}] + \mu^j$$

where:

- ▶ $\Gamma_{k'} = \kappa \left(\frac{k'}{k} - 1 \right)$ is the marginal adjustment cost
- ▶ $\text{MPK}^j = 1 - \delta + \zeta A_{\text{tfp}} \exp(z^j) (k^j)^{\zeta-1} - \frac{\partial \Gamma}{\partial k}$
- ▶ μ^j is the **irreversibility multiplier**
- ▶ The term $(1 - \delta) \mu^{j'}$ captures the **shadow value** if the constraint binds tomorrow

Euler Equation: Relative Error Form

Rearranging the Euler equation into **relative error** form:

$$\frac{\beta \mathbb{E}[\lambda' \cdot \text{MPK}^{j'} - (1 - \delta) \mu^{j'}] + \mu^j}{\lambda (1 + \Gamma_{k'})} - 1 = 0$$

Why relative error?

- ▶ All N Euler equations are **scale-free**
- ▶ Residuals can be interpreted as **percentage errors** in the optimality condition
- ▶ A residual of 10^{-3} means the agent is off by 0.1%

Aggregate Resource Constraint

All output must be allocated to consumption, investment, and adjustment costs:

$$\sum_{j=1}^N \left[\underbrace{Y^j + (1 - \delta) k^j}_{\text{resources}} - \underbrace{k^{j'}}_{\text{investment}} - \underbrace{\Gamma^j}_{\text{adj. cost}} - \underbrace{c^j}_{\text{consumption}} \right] = 0$$

- ▶ This is a **single** equation (not N separate ones)
- ▶ Under complete markets, goods flow freely between countries
- ▶ Consumption c^j is determined by λ and the Pareto weights

Complementarity Conditions

For each country j , the Fischer–Burmeister condition enforces irreversibility:

$$\text{FB}_\varepsilon(\mu^j, l^j) = \mu^j + l^j - \sqrt{(\mu^j)^2 + (l^j)^2 + \varepsilon^2} = 0$$

where $l^j = k^{j'} - (1 - \delta) k^j$.

Total system:

Equation	Count	Unknown
Euler equations	N	$k^{1'}, \dots, k^{N'}$
Aggregate resource constraint	1	λ
Fischer–Burmeister	N	$\mu^1, \dots, \mu^N$
Total	$2N + 1$	$2N + 1$

Summary: Equilibrium System

Well-posed system: $2N + 1$ equations in $2N + 1$ unknowns.

N	States	Policies	Equations
2	4	5	5
4	8	9	9
10	20	21	21
50	100	101	101
100	200	201	201

- ▶ Dimensions grow **linearly** in N —no combinatorial explosion
- ▶ Traditional grid methods: $\mathcal{O}(n^{2N})$ grid points
- ▶ DEQNs: a single neural network, trained via SGD

Finger Exercise: IRBC Euler Equations

Exercise

Consider the 2-country IRBC model. Suppose we **double** the capital depreciation rate δ from the baseline 0.01 to 0.02.

1. Without running any code: will the steady-state capital stock in each country *increase* or *decrease*? Why?
2. Will the optimal savings rate (investment / output) increase or decrease?
3. Open notebook `05_01_IRBC_DEQN_smooth.ipynb`, change δ , and verify your prediction.

Solution: IRBC Euler Equations

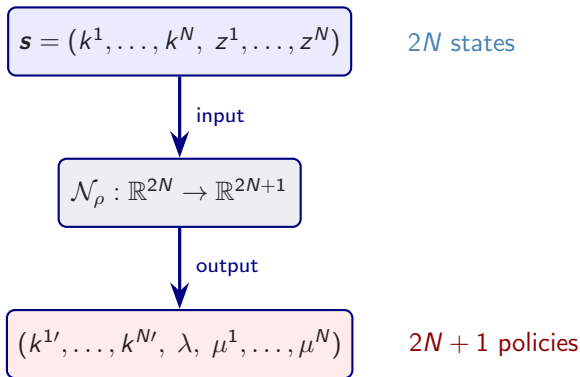
Answer

1. Steady-state capital **decreases** if A_{tfp} is held fixed at its $\delta = 0.01$ value: the Euler equation implies $r = 1/\beta - 1 + \delta$, so higher δ requires a higher marginal product of capital and (with diminishing returns) a lower k^* . **Caveat:** the script's calibration normalizes $A_{\text{tfp}} = (1/\beta - 1 + \delta)/\zeta$ so that $k_{\text{ss}} = 1$ at the baseline; if you re-derive A_{tfp} at the new δ , the normalized k_{ss} stays at 1 and the change shows up in A_{tfp} itself and in the consumption–investment split. The exercise asks for the A_{tfp} -fixed convention.
2. The savings rate **increases**. More capital depreciates each period, so a larger fraction of output must be reinvested just to maintain the (lower) capital stock.
3. In the notebook: changing δ from 0.01 to 0.02 should show the policy function shifting down (lower capital) and the investment share rising.

Part II

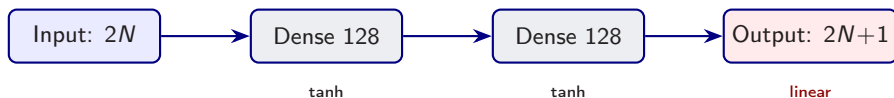
DEQN Formulation for the IRBC

State & Policy Variables



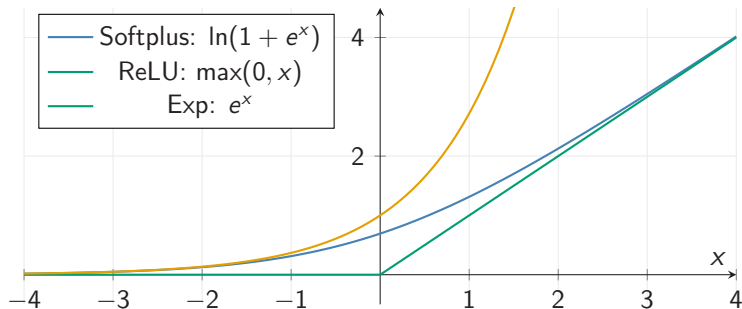
- ▶ The neural network maps the **full state** to **all policy variables simultaneously**
- ▶ For $N = 2$: input dimension 4, output dimension 5
- ▶ Outputs must satisfy economic constraints: a **linear** output layer is mapped through custom per-policy transforms

Neural Network Architecture



- ▶ **Hidden layers:** 2 layers $\times$ 128 units with **tanh** activation
- ▶ **Output layer:** **linear** (zero-initialized); raw outputs pass through custom transforms
- ▶ Transforms enforce the constraints: $k^{j'} > 0$ via $\exp \cdot \tanh$, $\lambda > 0$ via $\exp \cdot \tanh$, $\mu^j \geq 0$ via softplus (the smooth model omits μ^j)
- ▶ Total parameters: $(2N \cdot 128 + 128) + (128 \cdot 128 + 128) + (128 \cdot (2N + 1) + 2N + 1)$

Enforcing Positive, Feasible Outputs



The notebooks keep a **linear** output and apply bounded transforms: $k^{j'} = k^j \exp\{\bar{g} \tanh r\}$ and $\lambda = \lambda_{\text{ref}} \exp\{s \tanh r\}$ – the $\tanh$ caps the exponent, so no overflow (unlike a raw e^x) – while the inequality multipliers μ^j use **softplus**.

DEQN Loss Function for the IRBC

The loss is the **sum of squared residuals** across all equilibrium conditions; $\ell_\rho = 0$ iff every condition holds exactly.

Smooth benchmark (notebook 04a):

$$\ell_\rho^{\text{smooth}} = \frac{1}{B} \sum_{i=1}^B \left[\sum_{j=1}^N (\text{Euler}_j^{(i)})^2 + (\text{ARC}^{(i)})^2 \right]$$

Irreversibility extension (notebook 04b): add the Fischer–Burmeister block,

$$\ell_\rho^{\text{irrev}} = \ell_\rho^{\text{smooth}} + \frac{1}{B} \sum_{i=1}^B \sum_{j=1}^N (\text{FB}_j^{(i)})^2.$$

with B the batch size:

- ▶ $\text{Euler}_j = \frac{\beta \mathbb{E}[\lambda' \text{MPK}' - (1 - \delta)\mu'] + \mu}{\lambda(1 + \Gamma_{k'})} - 1$ (smooth: $\mu \equiv 0$)
- ▶ $\text{ARC} = \sum_j [Y^j + (1 - \delta)k^j - k^{j'} - \Gamma^j - c^j]$
- ▶ $\text{FB}_j = \mu^j + l^j - \sqrt{(\mu^j)^2 + (l^j)^2 + \varepsilon^2}$ (04b only)

Gauss–Hermite Quadrature

The Euler equation requires computing $\mathbb{E}[\cdot]$ over $N + 1$ normal shocks.

Gauss–Hermite quadrature in 1D:

$$\mathbb{E}[f(\varepsilon)] = \int f(\varepsilon) \phi(\varepsilon) d\varepsilon \approx \sum_{q=1}^Q w_q f(\varepsilon_q)$$

Tensor product for $N + 1$ dimensions:

$$\mathbb{E}\left[f(\varepsilon^1, \dots, \varepsilon^N, \varepsilon^{\text{agg}})\right] \approx \sum_{q_1=1}^Q \dots \sum_{q_{N+1}=1}^Q \left(\prod_d w_{q_d} \right) f(\varepsilon_{q_1}, \dots, \varepsilon_{q_{N+1}})$$

N	Shocks	Q	Quad. nodes
2	3	3	27
4	5	3	243
10	11	3	$\sim 177\text{K}$

What the companion notebooks do. Both 04a and 04b use the **Stroud-3 monomial rule** ($2(N + 1)$ nodes; 6 for $N = 2$), not the tensor-product Gauss–Hermite above; the latter is shown for contrast and is practical only at very low N .

Computing the Euler Expectation

Step-by-step for each quadrature node q :

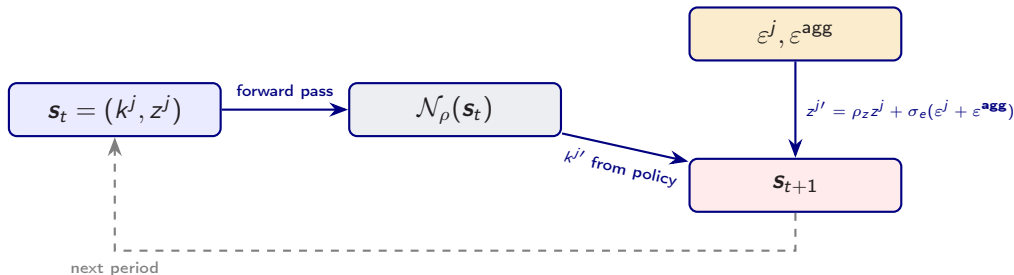
1. **Current state** $\mathbf{s}_t = (k^1, \dots, k^N, z^1, \dots, z^N)$
2. **Current policy** $\mathcal{N}_\rho(\mathbf{s}_t) \rightarrow (k^{1'}, \dots, k^{N'}, \lambda)$ (smooth, $N + 1$ outputs; the irreversible extension appends $\mu^1, \dots, \mu^N$)
3. **Next-period state** at node q :

$$k_q^{j'} = k^{j'}, \quad z_q^{j'} = \rho_z z^j + \sigma_\epsilon (\epsilon_q^j + \epsilon_q^{\text{agg}})$$

4. **Next-period policy**: evaluate $\mathcal{N}_\rho(\mathbf{s}_{t+1,q})$
5. **Accumulate**: E-term = $\sum_q w_q [\lambda'_q \cdot \text{MPK}_q^j - (1 - \delta) \mu_q^{j'}]$

Each quadrature node requires a **forward pass** through the network—but all nodes in a batch can be evaluated in parallel.

State Transition



- **Capital:** $k^{j'}$ is a policy output (endogenous)
- **TFP:** $z^{j'}$ evolves exogenously via AR(1) + shocks
- For simulation: draw $\varepsilon \sim \mathcal{N}(0, 1)$
- For expectations: use quadrature nodes ε_q

Training Algorithm

Algorithm 1: DEQN Training for IRBC (persistent simulation)

Input: Network $\mathcal{N}_\rho$, learning rate η , segments S , monomial nodes/weights
Initialize M trajectory heads $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(M)}$ from a feasible starting cloud
for segment $s = 1, \dots, S$ **do**
 Simulate: **for** $t = 1, \dots, T$, sample shocks and step each head one period under $\mathcal{N}_\rho$
 Flatten the $M \cdot T$ recorded states into a training batch
 for pass $p = 1, \dots, P$ **over mini-batches of the segment do**
 Compute loss ℓ_p on the segment (smooth: Euler + ARC; irreversible: also FB)
 Update: $\rho \leftarrow \rho - \eta \nabla_\rho \ell_p$ (Adam)
 end for
 Continue: $\mathbf{X}^{(m)} \leftarrow$ terminal state of trajectory m (*no reset to steady state*)
 Monitor: policy drift on $\mathbf{X}_{\text{anchor}}$; flag stable when below tolerance
end for
Diagnostic: zero-shock SSS check from dispersed feasible starts
Output: Trained $\mathcal{N}_{\rho^*}$

Scalability in N

N	States	Policies	NN params	Grid pts ($n=10$)
2	4	5	$\sim 4.8\text{K}$	10^4
4	8	9	$\sim 5.3\text{K}$	10^8
10	20	21	$\sim 6.9\text{K}$	10^{20}
50	100	101	$\sim 17\text{K}$	10^{100}
100	200	201	$\sim 30\text{K}$	10^{200}

- ▶ NN parameters grow **linearly** in N (only input/output layers scale)
- ▶ Grid methods grow **exponentially**: 10^{2N} points
- ▶ DEQN training cost per step: $\mathcal{O}(|\rho| \cdot B \cdot Q^{N+1})$
- ▶ For large N : replace tensor-product GH with monomial rules or QMC
- ▶ Theoretical underpinning: **deep ReLU networks** beat the curse of dimensionality for sparse-grid-representable functions (Montanelli & Du 2019)

From Brock–Mirman to IRBC

	Brock–Mirman	IRBC
Countries	1	N
States	(K, z)	$(k^1, \dots, k^N, z^1, \dots, z^N)$
Policies	C (or s)	$(k^{1'}, \dots, k^{N'}, \lambda, \mu^1, \dots, \mu^N)$
Equations	1 Euler	N Euler + 1 ARC + N FB
Constraints	none	irreversibility, adj. costs
Shocks	1	$N + 1$
Output act.	sigmoid	linear + transforms
Analytical sol.	yes ($\delta = 1$)	no

The IRBC is a natural **scaling test** for the DEQN methodology.

Recap: Three Building Blocks

Model

N -country IRBC with irreversible investment, adjustment costs, complete markets. $2N + 1$ equilibrium equations.

Network

$\mathcal{N}_\rho : \mathbb{R}^{2N} \rightarrow \mathbb{R}^{2N+1}$ with tanh hidden layers, linear output mapped through bounded transforms. All policies in one forward pass.

Training

Persistent simulation: one continuing trajectory ensemble, Adam on the sum of squared residuals.

⇒ **Next:** implementation in TensorFlow/Keras and results.

Part III

Implementation & Results

Notebook Overview

Three notebooks accompany this lecture (two in the L04 code/ folder, one in the L05 folder; self-contained, all equations inline):

1. 05_01_IRBC_DEQN_smooth.ipynb – *smooth benchmark, in-class walk-through (this lecture's code/)*
 - ▶ Parameters & steady state, monomial quadrature; Keras network, production/consumption/adjustment-cost equations
 - ▶ Cost function (Euler + ARC), persistent-simulation training loop
 - ▶ Time-invariance check and zero-shock stochastic steady state
2. 05_02_IRBC_DEQN_irreversible.ipynb – *irreversible-investment extension (this lecture's code/)*
 - ▶ KKT multipliers μ^j and feasibility-by-construction policy; Fischer–Burmeister complementarity loss
 - ▶ Same training loop and diagnostics as the smooth benchmark
3. 05_03_IRBC_Exercise.ipynb – *hands-on exercise*
 - ▶ Task 1: comparative statics on the deterministic steady state
 - ▶ Task 2: inverse-loss weighting for multi-component losses (ReLoBRaLo, from Session 3)

Parameters

Symbol	Name	Value	Description
β	Discount factor	0.99	Quarterly calibration
ζ	Capital share	0.36	Cobb–Douglas
δ	Depreciation	0.01	Low depreciation
ρ_z	TFP persistence	0.95	Highly persistent
σ_e	Shock std. dev.	0.01	Small shocks
κ	Adjustment cost	0.50	Moderate costs
$\gamma_{\min}$	Min IES	0.25	High risk aversion
$\gamma_{\max}$	Max IES	1.00	Log utility
k_{ss}	Steady-state capital	1.00	Normalization

Pareto weights: $\tau^j = (A_{\text{tfp}} - \delta)^{1/\gamma_j} = c_{ss}^{1/\gamma_j}$. At the symmetric steady state $c_{ss} = A_{\text{tfp}} - \delta$ for every country, so $\lambda_{ss} = \tau^j (c_{ss})^{-1/\gamma_j} = 1$ shared across all j regardless of heterogeneous γ_j .

Steady State

At steady state ($z^j = 0$, $k^j = k_{ss} = 1$, $\mu^j = 0$):

$$A_{\text{tfp}} = \frac{1 - \beta(1 - \delta)}{\zeta\beta} = \frac{1 - 0.99 \cdot 0.99}{0.36 \cdot 0.99} \approx 0.0559$$

$$Y_{ss} = A_{\text{tfp}} \cdot k_{ss}^{\zeta} = A_{\text{tfp}} \approx 0.0559$$

$$I_{ss} = \delta \cdot k_{ss} = 0.01$$

$$c_{ss} = Y_{ss} - I_{ss} \approx 0.0459$$

Verification: the ARC should be satisfied: $\sum_j [Y_{ss} - I_{ss} - c_{ss}] = 0 \checkmark$

The Euler equation at steady state gives the MPK $= 1/\beta$, which pins down A_{tfp} .

Network Definition (Keras)

```
N_COUNTRIES = 2
n_states = 2 * N_COUNTRIES          # = 4
n_policies = 2 * N_COUNTRIES + 1    # = 5

nn = keras.Sequential([
    keras.layers.Dense(128, activation='tanh',
                        input_shape=(n_states,)),
    keras.layers.Dense(128, activation='tanh'),
    keras.layers.Dense(n_policies, activation=None,
                        kernel_initializer='zeros')
])
```

- ▶ tanh hidden activations; output layer is linear (zero-initialized)
- ▶ Raw outputs pass through custom transforms (exp·tanh, sigmoid, softplus) to enforce the constraints
- ▶ Output: $[k^{1'}, k^{2'}, \lambda, \mu^1, \mu^2]$

Production & Consumption

Production:

```
def production(k, z):  
    return A_tfp * tf.exp(z) \  
           * k**zeta
```

$$Y^j = A_{\text{tfp}} \exp(z^j) (k^j)^\zeta$$

Consumption (from FOC):

```
def consumption(lamb, j):  
    tau = (A_tfp - delta)**(1./  
           gammas[j])  
    return (lamb/tau)**(-gammas[j])
```

$$c^j = (\lambda/\tau^j)^{-\gamma_j}$$

All functions are vectorized over the batch dimension using TensorFlow operations.

Adjustment Costs

```
def adj_cost(k, kp):  
    j = kp / k - 1.0  
    return 0.5 * kappa * j**2 * k
```

```
def adj_cost_kp(k, kp):  
    return kappa * (kp / k - 1.0)
```

```
def adj_cost_k(k, kp):  
    j = kp / k - 1.0  
    return -0.5 * kappa * j * (j + 2.0)
```

Marginal product of capital:

```
def mpk(k, z, kp):  
    Fk = zeta * A_tfp * tf.exp(z) \  
        * k**(zeta - 1)  
    return 1 - delta + Fk - adj_cost_k(k, kp)
```

Fischer–Burmeister Function

```
def fischer_burmeister(mu, I, eps=1e-4):  
    return mu + I - tf.sqrt(mu**2 + I**2 + eps**2)
```

Investment:

```
def investment(k, kp):  
    return kp - (1.0 - delta) * k
```

The FB function is:

- ▶ **Differentiable** everywhere for $\varepsilon > 0$ (unlike $\min(\mu, I)$)
- ▶ **Exact zero set** at $\varepsilon = 0$: $\mu \geq 0, I \geq 0, \mu \cdot I = 0$
- ▶ The small ε prevents division-by-zero at the origin

Cost Function Structure

```
@tf.function
def compute_cost(states, nn):
    policies = nn(states)
    # Extract  $k'$ ,  $\lambda$ ,  $\mu$  per country
    # For each GH quadrature node:
    #     Compute next state, evaluate  $nn$ ,
    #     accumulate Euler integrand
    # Euler residual (relative error)
    # ARC: sum resources - uses = 0
    # FB: complementarity per country
    loss = mean(euler**2 + arc**2 + fb**2)
    return loss
```

Key point: the entire computation graph is inside `@tf.function` for **automatic differentiation** via GradientTape.

Persistent-Simulation Training

The companion notebooks train on **one continuing trajectory ensemble** – no Phase 1 / Phase 2 switch, no resets to steady state.

- ▶ Maintain $M = \text{N_TRAJECTORIES}$ stochastic trajectory heads $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(M)}$.
- ▶ Each **training segment**: simulate the heads forward for $T = \text{SIMULATION_LENGTH}$ stochastic periods under the *current* policy network; flatten into a training cloud of size $M \cdot T$; perform a fixed number of SGD passes.
- ▶ Continue from the segment's terminal states: $\mathbf{X}_{\text{start}} \leftarrow \mathbf{X}_{\text{end}}$. The trajectory ensemble **co-evolves** with the policy.
- ▶ A `SAMPLING_MODE` switch (simulation vs exogenous) is available for ablations; the default `simulation` mode never sees a uniform burn-in.

Why this works without a separate burn-in.

- ▶ The policy parameterization is feasibility-bounded by construction:
 - ▶ smooth (04a): $k^{j'} = k^j \exp\{\bar{g} \tanh r_j(\mathbf{s})\}$ keeps $k^{j'} > 0$;
 - ▶ irreversible (04b): $k^{j'} = (1 - \delta)k^j + l^j$ with $l^j \geq 0$ enforced via a sigmoid head.
- ▶ Random initial weights cannot drive the simulation into infeasible states.

Diagnostics: Time-Invariance and Zero-Shock SSS

Time invariance. The architecture has no calendar-time input, so any fixed weight vector defines a stationary recursive policy. The *numerical* question is whether SGD has stopped moving the policy function.

- ▶ Evaluate the policy on a fixed anchor cloud X_{anchor} after each monitoring interval.
- ▶ Report `policy_drift_rms` and `policy_drift_max`.
- ▶ When both fall below `TIME_INVARIANCE_TOL_RMS / _MAX`, the run is flagged **stable**.

Zero-shock stochastic steady state (SSS). Iterate the learned policy from `ZERO_SHOCK_N_STARTS` dispersed feasible starts with $\varepsilon_t^j = \varepsilon_t^{\text{agg}} = 0$.

- ▶ All paths should converge to a common point.
- ▶ In a sensible SSS: $\dot{I}^j \approx \delta k^j$ (investment replaces depreciation) and, in 04b, the irreversibility multiplier $\mu^j \approx 0$.

Persistent-Simulation Training: What to Monitor

With a single training pipeline (no Phase 1/Phase 2 switch), *four* signals carry the whole convergence story:

1. **Total loss** decays monotonically across training segments (the *loss-vs-segment* panel in the notebooks).
2. **Per-residual mean magnitudes** – `mean_abs_euler`, `mean_abs_arc`, and (in 04b) `mean_abs_fb` – all fall below the prescribed tolerance.
3. **Policy drift on the anchor cloud** crosses from **moving** to **stable**: only then is the policy time-invariant in the recursive sense.
4. **Zero-shock SSS** converges from dispersed feasible starts to a common point with $l^j \rightarrow \delta k^j$ and (in 04b) $\mu^j \rightarrow 0$.

Historical note. An earlier four-approach comparison (uniform vs simulation-only vs two-phase vs ReLoBRaLo) on the same model is preserved in `figures/irbc_4approach_loss.pdf`; it is *not* produced by the current notebooks, which run a single persistent-simulation pipeline.

Convergence: Per-Equation Residuals

After training, we evaluate residuals on a test cloud (smooth + irreversible):

Residual	Mean ·	Max ·
Euler (country 1)	$\sim 10^{-4}$	$\sim 10^{-3}$
Euler (country 2)	$\sim 10^{-4}$	$\sim 10^{-3}$
ARC	$\sim 10^{-4}$	$\sim 10^{-3}$
<i>Irreversibility extension (notebook 04b only):</i>		
FB (country 1)	$\sim 10^{-5}$	$\sim 10^{-4}$
FB (country 2)	$\sim 10^{-5}$	$\sim 10^{-4}$

- ▶ Euler residuals $< 10^{-3} \Rightarrow$ policy is accurate to within 0.1%
- ▶ FB residuals near zero $\Rightarrow$ complementarity is well satisfied (04b only)
- ▶ These are **relative errors**—interpretable as percentage deviations

Policy Functions

After training, we visualize the learned policies:

- ▶ **Capital policy** $k^{j'}$ vs. k^j :
 - ▶ Near 45-degree line (close to replacement)
 - ▶ Higher investment when TFP is high
- ▶ **Shadow price** λ vs. z^j :
 - ▶ Decreasing in productivity (more output $\Rightarrow$ lower scarcity)
- ▶ **Multiplier** μ^j vs. k^j :
 - ▶ Near zero when k is low (investment is positive)
 - ▶ Positive when k is high (irreversibility binds)
- ▶ **Investment** I^j vs. k^j :
 - ▶ Positive most of the time
 - ▶ Occasionally zero (constraint active)

Economic Diagnostics

Consumption sharing:

- ▶ Under complete markets, c^1 and c^2 should be **positively correlated**
- ▶ Perfect risk sharing: c^1/c^2 is constant
- ▶ With heterogeneous γ : ratio varies with λ

Reconnects with the **consumption-correlation puzzle** from slide 7: complete markets predict $\rho(c^1, c^2) \approx 1$, but in data $\rho(c^1, c^2) < \rho(y^1, y^2)$; closing the gap needs incomplete markets or frictions.

Trade balance:

- ▶ $TB^j = Y^j - c^j - I^j - \Gamma^j$ (> 0 when j exports)
- ▶ Sums to zero across countries

Ergodic distribution:

- ▶ Scatter plot of (k^1, k^2) over long simulation
- ▶ Should concentrate near (k_{ss}, k_{ss})
- ▶ Correlation structure reveals risk-sharing patterns

Complementarity check:

- ▶ Plot μ^j vs. I^j
- ▶ Should lie on the axes: $\mu = 0$ or $I = 0$
- ▶ Fraction of time constraint binds

Validation: Euler LHS vs. RHS

Key diagnostic: compare both sides of the Euler equation on a test set.

LHS: $\lambda \cdot (1 + \Gamma_{k'})$

RHS: $\beta \mathbb{E}[\lambda' \cdot \text{MPK}' - (1 - \delta)\mu'] + \mu$

Targets:

- ▶ Points should lie on the **45-degree line**
- ▶ Mean relative error $< 10^{-3}$
- ▶ Max relative error $< 10^{-2}$

Also check:

- ▶ ARC residual ≈ 0 across all states
- ▶ $\mu^j \approx 0$ whenever $I^j > 0$
(complementarity)
- ▶ $I^j \geq 0$ everywhere (irreversibility)

These diagnostics confirm the network has learned a valid **approximate equilibrium**.

Bridge: from solving to estimating

What we leave with today. An IRBC DEQN trained with a single persistent-simulation pipeline and equal loss weights: the same recipe scales from $N = 2$ to $N = 10$ countries with no architectural change. NAS-tuned width and ReLoBRaLo-balanced loss components, from Session 3 this afternoon, are the refinements to reach for when the equal-weight recipe stalls.

Day 1, in one line.

- ▶ The equilibrium residual *is* the loss (Session 2).
- ▶ Architecture and loss weights are the two knobs that decide whether training works (Session 3).
- ▶ Cohorts and constraints do not change the principle (Session 4); neither does a $2N$ -dimensional state (today).

Tomorrow.

- ▶ Solving is not the end: a trained solver is expensive to re-run. Deep surrogates and Gaussian processes *amortize* it (Session 6).
- ▶ That makes structural estimation tractable (Session 7).
- ▶ Then the same residual idea in continuous time: PDEs, HJB, and the coupled HJB–KFE system

Questions?

Simon Scheidegger

HEC, University of Lausanne

<https://sischei.github.io>

Materials: https://github.com/sischei/deep_learning_padova_0926

Next up:

AutoDiff → Sequence Space →
OLG → Heterogeneous Agents
(Young's method)